

```
cd /notebooks/storage/project/ocgan-modernized
source .venv/bin/activate
```

```
#!/usr/bin/env bash
```

```
set -euo pipefail
```

```
cd /notebooks/storage/project/ocgan-modernized
```

```
source .venv/bin/activate
```

```
rm -rf /notebooks/.Trash-0
```

```
mkdir -p /notebooks/.Trash-0/files /notebooks/.Trash-0/info
```

```
du -sh /notebooks/.Trash-0 /notebooks 2>/dev/null
```

```
df -h
```

```
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
```

Metto tutto in un piano cronologico unico per il binario B, dalle fondamenta fino alle estensioni più ambiziose, includendo anche i perfezionamenti importabili dagli altri binari solo se compatibili con questo path.

Ragionato per 16s

Perfetto. Da qui in poi ci concentriamo **solo sul Binario B: OCGAN modernizzato**.

La logica del path è questa: non difendere OCGAN “puro”, ma **tenere solo ciò che è ancora utile** — soprattutto branch ricostruttiva, latent regularization come idea, informative-negative mining — e innestarlo dentro una pipeline moderna basata su **backbone pretrained, score multipli, anomalie sintetiche, branch discriminativa, memory signals e teacher-student discrepancy**.

Questa scelta è coerente sia con il cuore del paper OCGAN, che usa AE denoising, latent constraint, visual discriminator, classifier e informative-negative mining, sia con la traiettoria successiva della letteratura, dove PaDiM, PatchCore, DRAEM e RD4AD hanno mostrato che feature pretrained, patch modeling, synthetic anomalies e reverse distillation sono molto forti sui benchmark moderni.

fsfs

Qui sotto trovi il **programma cronologico completo**, ordinato nel modo in cui conviene davvero eseguirlo. Ho incluso tutti i perfezionamenti percorribili su questo path, anche quelli importati dai discorsi precedenti, ma solo se sono compatibili con Binario B.

1. Fase 0 — Definire il target del metodo prima di scrivere codice

Prima di implementare, fissa esplicitamente il modello obiettivo. Il sistema finale non sarà “OCGAN classico con qualche ritocco”, ma un ibrido con questi blocchi: **encoder pretrained**, **branch ricostruttiva residuale**, **latent compactness o one-class bottleneck**, **hard-negative mining evoluto**, **synthetic anomalies**, **branch discriminativa su input e ricostruzione**, **feature discrepancy**, **score composito**, e possibilità di aggiungere **memory-bank signal** e **teacher-student discrepancy**. Questo è il modo corretto di conservare l'intuizione di OCGAN evitando il limite del GAN pixel-centric come motore principale. OCGAN stesso nasce per vincolare il latent space e migliorare l'AE classico; i lavori successivi mostrano che i segnali migliori oggi arrivano spesso dalle feature pretrained e dai modelli patch-based o teacher-student.

fsfs

Decisioni da fissare subito:

1. il path principale usa **PyTorch**;
2. la prima famiglia di backbone è **ResNet50 / WideResNet50**;
3. il training userà da subito **validation separata**, **multi-run**, **score composito** e **benchmark industriale vero**, non solo MNIST-like;
4. OCGAN storico verrà usato solo come **baseline di controllo interna**, non come obiettivo finale. Il repo TensorFlow attuale è troppo minimale e datato per sostenere questo tipo di ricerca. (github.com)

2. Fase 1 — Rifondazione del repository e dell'infrastruttura

Questa è la prima cosa da fare davvero. Se la salti, tutto il resto produce risultati poco affidabili.

2.1 Porting e struttura del progetto

Porta tutto in **PyTorch** con una struttura modulare: `configs`, `datasets`, `models`, `losses`, `miners`, `scorers`, `metrics`, `trainers`, `callbacks`, `scripts`. Devi poter attivare o spegnere facilmente **backbone pretrained**, **branch ricostruttiva**, **compactness loss**, **synthetic anomalies**, **discriminative head**, **memory score** e **teacher-student score** senza riscrivere il trainer. Questo è essenziale perché il progetto richiederà molte ablation incrociate. Il repo TF e quello MXNet non sono progettati per questo livello di modularità. (github.com)

fsfs

2.2 Reproducibility hardening

Integra subito:

- seed globali;
- determinismo controllabile;
- salvataggio completo delle config;
- checkpoint top-k;
- resume robusto;
- logging di ambiente e hardware;
- multi-run automatico per seed e classe.

Questo è obbligatorio perché il repo TensorFlow dichiara esplicitamente che i risultati MNIST sono stati misurati una sola volta per cifra, che è troppo fragile per qualsiasi conclusione seria. (github.com)

2.3 Training utilities

Aggiungi subito:

- mixed precision;
- gradient accumulation;
- gradient clipping;
- EMA dei pesi;
- supporto multi-GPU;
- profiler;
- smoke tests;
- test NaN/Inf;
- overfit test su mini-dataset.

Sono tutte migliorie percorribili sul path B e hanno costo basso-medio rispetto al guadagno in velocità e stabilità.

2.4 Logging obbligatorio

Il training deve loggare:

- reconstruction loss;
- perceptual / structural loss;
- latent compactness loss;
- discriminative loss;
- eventuale adversarial loss;
- AUROC, AUPRC, F1;
- tempo per epoca, throughput, VRAM;
- score histograms normali vs anomali;
- reconstructions;
- anomaly maps;
- esempi di hard negatives.

Questa parte non alza direttamente l'AUROC, ma è ciò che permette di capire cosa sta davvero funzionando.

3. Fase 2 — Protocollo sperimentale serio e definitivo

Questo è il secondo blocco da chiudere prima di fare tuning aggressivo.

3.1 Split corretti

Usa sempre quattro split:

- `train-normal`
- `val-normal`
- `val-mixed`
- `test-blind`

La `val-mixed` serve per scegliere soglie, checkpoint e iperparametri. Non va mai usato il test per threshold o tuning. OCGAN nel paper distingue due protocolli di valutazione per evitare confronti scorretti; il tuo setup deve essere ancora più rigoroso.

fsfs

3.2 Compatibilità storica solo come controllo

Replica protocollo 1 e protocollo 2 di OCGAN solo come benchmark storico interno, non come protocollo principale del progetto. Il paper definisce esplicitamente protocollo 1 con split 80/20 in-class e test bilanciato con out-of-class, e protocollo 2 con gli split nativi del dataset.

fsfs

3.3 Anti-leakage

Metti in pipeline:

- deduplica esatta;
- near-duplicate detection;
- split per oggetto/istanza/lotto quando applicabile;
- statistiche di normalizzazione calcolate solo sul train;
- verifica contaminazione del nominale.

Su anomaly detection basta poco leakage per falsare tutto.

3.4 Multi-run obbligatorio

Stabilisci subito:

- 10 seed su MNIST/Fashion-MNIST;
- 5–10 seed su CIFAR10;
- almeno 5 seed su MVTec AD e MVTec AD 2.

Riporta media, deviazione standard e risultati per classe. Questo corregge direttamente la fragilità dei repo originali e trasforma il progetto in ricerca seria. (github.com)

3.5 Metriche

Per image-level: AUROC, AUPRC, F1 su validation threshold, FPR@95TPR.

Per localization: pixel AUROC, PRO/AU-PRO, Dice/F1.

Questo è importante perché MVTec AD è pensato sia per detection sia per localizzazione, con ground truth pixel-precise. (mvttec.com)

3.6 Model selection

Abbandona la selezione via sola validation MSE. OCGAN la usa, ma sul path B devi scegliere i checkpoint con una metrica composita basata su separazione tra normali e pseudo-anomali nella validation mista, qualità della localizzazione e stabilità del training.

fsfs

4. Fase 3 — Benchmark plan e dataset order

L'ordine giusto dei dataset è:

4.1 Primo giro: MNIST, Fashion-MNIST, CIFAR10

Servono per:

- verificare che la pipeline funzioni;
- controllare il mining;
- fare ablation veloci;
- mantenere confrontabilità con OCGAN. OCGAN stesso mostra che CIFAR10 è sensibilmente più difficile dei dataset più allineati.

fsfs

4.2 Secondo giro: MVTec AD

Questo è il vero banco di prova del path B. MVTec AD è il benchmark industriale di riferimento, con 15 categorie, immagini ad alta risoluzione, train defect-free e test misto con annotazioni pixel-level. (mvttec.com)

4.3 Terzo giro: MVTec AD 2

Usalo quando il sistema è già stabile. MVTec AD 2 è pensato per scenari più difficili e meno saturi, quindi è perfetto per capire se il path modernizzato regge davvero. (mvttec.com)

4.4 Few-shot e robustness

Dopo la baseline forte su MVTec AD, aggiungi:

- curve 1%, 5%, 10%, 25%, 50%, 100% di train nominale;
- stress test su blur, illuminazione, compressione, misalignment e background shift.

PatchCore dichiara risultati competitivi anche nel few-samples regime, quindi questo è un confronto particolarmente rilevante.

5. Fase 4 — Pipeline immagini e preprocessing

Questa fase va fatta prima del tuning pesante del modello, perché su industrial AD il preprocessing cambia davvero i risultati.

5.1 Resize e crop

Implementa fin da subito:

- resize con aspect ratio preservato;
- anti-aliased resizing;
- pad invece di distorsione quando serve;
- center crop controllato;
- tiling/patching per alta risoluzione;
- multi-scale inference come opzione.

Questo è particolarmente importante su MVTec AD e AD 2, dove gli oggetti sono ad alta risoluzione e i difetti possono essere locali e piccoli. (mvtectec.com)

5.2 Normalizzazione

Se usi backbone pretrained, normalizza nel modo atteso dal backbone. Se alleni una branch da zero, usa mean/std calcolati solo sul train. Tieni come ablation il confronto con scaling $[-1, 1]$ solo per continuità con OCGAN.

fsfs

5.3 Opzioni dominio-specifiche

Tieni implementabili ma non attivare tutto subito:

- CLAHE;
- contrast normalization;
- foreground masking;
- background removal;
- alignment/pose normalization;

- crop strategy guidata dall'oggetto;
- grayscale vs RGB.

Sono tutte migliori percorribili, ma vanno attivate solo dove il dominio lo giustifica.

5.4 Data cleaning

Fai prima possibile:

- exact duplicate removal;
- near-duplicate removal;
- audit delle immagini nominali contaminate;
- rimozione immagini corrotte.

Questo è un investimento a basso costo e alto valore.

6. Fase 5 — Augmentations conservative

Qui la regola è: preservare la normalità.

6.1 Base augmentations

Attiva:

- piccole traslazioni;
- leggere rotazioni;
- contrasto/luminosità moderati;
- gaussian noise controllato;
- blur lieve.

6.2 Industrial augmentations

Per MVTec-like:

- jitter di illuminazione;
- piccole variazioni di camera/acquisizione;
- lievi shift geometrici;
- blur lieve.

6.3 Cosa NON usare subito

- elastic deformation forte;
- cutout aggressivo nel training finale;
- augmentation che cambiano la semantica del nominale.

Queste possono stare come pretraining o robustness tests, ma non come default.

7. Fase 6 — Backbone pretrained e feature spine

Questa è una delle svolte principali del path B.

7.1 Prima scelta

Parti con:

- ResNet50
- WideResNet50

PaDiM usa una CNN pretrained per patch embedding; PatchCore usa embeddings da modelli ImageNet con memory bank; RD4AD usa teacher encoder pretrained. Quindi il backbone pretrained non è un optional, è uno dei cardini delle linee forti post-2019.

7.2 Strategia di training

Ordine consigliato:

1. backbone frozen;
2. solo quando tutto è stabile, partial fine-tuning;
3. adapter-only tuning come terza variante.

7.3 Feature discrepancy

Aggiungi subito score e loss basati su differenza fra feature di input e feature della ricostruzione. Questo è uno dei segnali più utili importabili dai metodi moderni senza trasformare il progetto in qualcos'altro.

8. Fase 7 — Branch ricostruttiva modernizzata

Qui non stai più costruendo un autoencoder datato in stile 2019, ma un ricostruttore moderno e controllato.

8.1 Architettura

Usa:

- residual blocks;
- anti-aliased downsampling;
- upsample+conv;

- multi-scale outputs opzionali;
- 1–2 blocchi di attenzione solo se servono;
- skip leggere o gated, non U-Net piena.

OCGAN parte da un denoising autoencoder con bottleneck e tanh-bounded latent; l'idea da tenere è la branch ricostruttiva, non l'architettura originaria.

fsfs

8.2 Modalità di ricostruzione

Supporta:

- reconstruction classica;
- denoising reconstruction;
- masked reconstruction;
- corruption-aware reconstruction.

8.3 Skip connections

Usale solo in forma controllata. Skip troppo forti rischiano di far passare le anomalie invece di sopprimerle.

9. Fase 8 — Loss di ricostruzione moderne

Questa è una delle prime migliorie che va implementata subito.

9.1 Loss stack iniziale

Usa come baseline forte:

- L1
- MS-SSIM
- perceptual loss

La sola MSE è troppo debole, tende a oversmootare e perde dettagli fini. Questo è particolarmente rilevante quando le anomalie sono sottili o strutturali. OCGAN usa una ricostruzione MSE-centrica; sul path B questa scelta va superata.

fsfs

9.2 Loss opzionali

Tieni disponibili:

- gradient loss;

- frequency-domain loss;
- feature matching;
- LPIPS-like discrepancy.

Queste vanno introdotte solo se noti che il modello perde dettagli ad alta frequenza o texture deboli.

10. Fase 9 — Latent regularization moderna

Questa è la parte del vecchio OCGAN che va mantenuta come principio, non come forma rigida.

10.1 Cosa tenere

Tieni l'idea che il bottleneck debba essere **compatto e nominale**, cioè non una semplice compressione libera.

10.2 Cosa cambiare

Non usare più come centro del progetto il solo prior uniforme $\mathcal{U}(-1, 1)^d$. Quello resta come baseline storica. Il path B deve puntare su:

- center loss / latent compactness;
- one-class bottleneck;
- Deep-SVDD-like compactness;
- eventualmente hyperspherical latent;
- eventualmente mixture prior;
- energy-based latent score.

Questa direzione è anche concettualmente vicina a RD4AD, che introduce un one-class bottleneck embedding per il teacher-student path.

10.3 Ordine corretto

Prima:

1. center/compactness loss;
2. one-class bottleneck;
3. latent dim sweep.

Solo dopo:

4. hypersphere;
 5. mixture prior;
 6. flow sul latent.
-

11. Fase 10 — Hard-negative mining modernizzato

Questa è probabilmente la migliore eredità diretta di OCGAN.

11.1 Prima implementazione

Fai subito:

- mining multi-step;
- PGD-style ascent;
- warmup prima di attivarlo;
- replay buffer di hard negatives;
- mining guidato da score composito.

Il paper OCGAN usa gradient-descent based sampling nel latent per trovare punti che generano esempi potenzialmente out-of-class; questa idea va conservata, ma resa molto più forte.

fsfs

11.2 Seconda ondata

Aggiungi:

- multi-start;
- diversity filter;
- boundary-aware sampling;
- hard negative replay persistente;
- contrastive push-away fra normali e negatives.

11.3 Mining oltre il latent

Questa è una parte essenziale del path B: il mining non deve restare confinato nel latent classico. Devi introdurre:

- feature-space hard negatives;
- negatives guidati da feature discrepancy;
- negatives guidati da teacher gap;
- negatives guidati da memory distance.

Questa è una delle fusioni più forti con la letteratura moderna.

12. Fase 11 — Synthetic anomalies

Questa fase va costruita subito dopo che il backbone e il mining funzionano.

12.1 Pixel-space anomalies

Implementa:

- CutPaste;
- Perlin masks;
- texture overlays;
- copy-paste locale;
- blur locale;
- scratches/stains sintetici.

DRAEM mostra precisamente che anomaly synthesis + reconstruction/discrimination è una direzione molto potente per anomaly detection e localization.

12.2 Feature-space anomalies

Questa è ancora più importante nel path B:

- gaussian perturbations sulle feature;
- feature mixing;
- adapter-space anomaly generation;
- patch-feature corruption.

È la linea più vicina a SimpleNet, che modernizza in pratica la nozione di informative negatives. (arxiv.org)

12.3 Usi corretti

Le anomalie sintetiche vanno usate per:

- branch discriminativa;
- validation mixed;
- anomaly maps;
- mining curriculum;
- robustness checks.

13. Fase 12 — Branch discriminativa moderna

Qui entra il blocco più vicino a DRAEM.

13.1 Struttura

Costruisci una testa discriminativa che riceve:

- input;
- ricostruzione;

- eventualmente differenza o concatenazione;
- eventualmente feature gap.

L'obiettivo non è solo classificare “anomalo o no”, ma produrre anche una **anomaly map densa**.

13.2 Perché è cruciale

Il reconstruction error puro spesso non basta. DRAEM dimostra il valore di una rete discriminativa che osserva input e ricostruzione “normalizzata” per localizzare dove stanno le discrepanze. ([semanticscholar.org](https://www.semanticscholar.org))

13.3 Heads da tenere

Supporta:

- image-level head;
 - patch-level head;
 - dense anomaly map head.
-

14. Fase 13 — Memory-like scoring compatibile con Binario B

Anche se il binario scelto non è l'ibrido C, nel path B puoi comunque importare una parte importante della logica PatchCore-like.

14.1 Cosa introdurre

Senza trasformare il modello in un puro PatchCore, aggiungi:

- patch feature extraction dal backbone;
- nearest-neighbor distance rispetto a un reference set nominale;
- eventuale memory subset / coreset.

PatchCore usa proprio una memory bank rappresentativa di patch nominali e ottiene risultati molto forti su MVTec AD, quindi questo segnale va importato nel path B anche se non fai ancora un binario C dedicato.

14.2 Come usarlo nel path B

Usalo come:

- score aggiuntivo;
- guida per il mining;
- supporto alla localizzazione;
- termine di calibrazione del final score.

15. Fase 14 — Teacher-student discrepancy leggera nel path B

Anche qui: non stai ancora facendo un binario C puro tipo RD4AD, ma puoi importarne il segnale più utile.

15.1 Cosa aggiungere

Introduci una variante leggera con:

- teacher pretrained frozen;
- student/decoder ricostruttivo;
- discrepancy score tra feature teacher e feature ricostruite.

RD4AD mostra che il reverse distillation da one-class embedding è molto forte, soprattutto perché usa una teacher encoder frozen e uno student decoder che ricostruisce rappresentazioni multiscala.

15.2 Come tenerlo nel path B

All'inizio tienilo come **score e loss ausiliari**, non come architettura dominante. Solo dopo, se funziona molto bene, può diventare una parte più centrale.

16. Fase 15 — Stabilizzazione training e ottimizzazione

Questa fase va attivata presto, non alla fine.

16.1 Ottimizzatori

Parti con:

- AdamW come default;
- Adam come controllo;
- RMSProp solo come baseline storica;
- Lion solo come ablation secondaria.

16.2 Scheduler

Usa:

- warmup;

- cosine decay;
- ReduceLROnPlateau come alternativa;
- one-cycle solo per test rapidi.

16.3 Stabilizzazione

Attiva:

- EMA dei pesi;
- gradient clipping;
- curriculum training;
- attivazione ritardata di moduli più aggressivi;
- batch-size effects study.

16.4 Se mantieni componenti adversarial

Se nel path B resta una parte avversaria, usa da subito:

- hinge loss;
- spectral normalization;
- R1 regularization;
- TTUR;
- update ratio dei moduli come iperparametro.

Queste sono tutte migliorie piccole ma ad alta probabilità di beneficio.

17. Fase 16 — Scoring finale, thresholding e calibrazione

Questa è una parte chiave del path B. Non usare mai più un solo score.

17.1 Score da combinare

Il final score deve poter includere:

- reconstruction residual;
- perceptual discrepancy;
- latent abnormality;
- discriminative anomaly score;
- patch-nearest-neighbor score;
- teacher-student discrepancy;
- anomaly map aggregation.

17.2 Fusione

Parti con:

- normalizzazione robusta degli score;
- weighted linear fusion;
- rank fusion.

Poi prova:

- logistic regression fusion sulla validation mixed.

17.3 Thresholding

Implementa:

- soglia da validation mixed;
- soglia per FPR target;
- EVT/GPD tail fitting come ablation avanzata;
- eventuale temperature scaling.

Questa fase è anche quella che rende il sistema più deployable.

18. Fase 17 — Robustezza, few-shot, debugging avanzato

Quando il modello è competitivo su benchmark standard, fai queste estensioni.

18.1 Robustness tests

Stress test su:

- blur;
- illuminazione;
- JPEG;
- background shift;
- misalignment;
- camera changes.

18.2 Few-shot

Valuta:

- 1%, 5%, 10%, 25%, 50%, 100% di nominali.

Questo serve per capire se la parte pretrained + synthetic anomalies + score composito regge davvero in regime povero di dati.

18.3 Failure analysis

Analizza:

- classi o categorie peggiori;
 - failure cases ricorrenti;
 - anomalie sottili vs strutturali;
 - casi in cui reconstruction e feature score si contraddicono.
-

19. Fase 18 — Ablation roadmap finale

L'ordine corretto delle ablation è questo.

19.1 Blocco 1

- backbone pretrained frozen
- branch ricostruttiva residuale
- L1 + MS-SSIM + perceptual
- score composito base

19.2 Blocco 2

- latent compactness
- mining multi-step
- synthetic anomalies pixel-space
- branch discriminativa input+reconstruction

19.3 Blocco 3

- feature-space anomalies
- feature discrepancy
- patch NN score
- teacher-student discrepancy

19.4 Blocco 4

- partial fine-tuning
- anomaly map improvements
- calibration/thresholding
- robustness/few-shot

19.5 Blocco 5

- hypersphere / mixture prior
- stronger attention

- additional memory compression
 - advanced flow-based latent options
-

20. Ordine pratico finale “to-do” senza salti

Se vuoi la sequenza più concreta possibile, è questa:

1. rifai il repo in PyTorch;
2. imposti protocollo moderno con validation mixed;
3. replichi una baseline OCGAN-derived minima solo per controllo;
4. attivi backbone pretrained frozen;
5. costruisci branch ricostruttiva residuale;
6. sostituisci MSE con L1 + MS-SSIM + perceptual;
7. introduci latent compactness / one-class bottleneck;
8. attivi score composito base;
9. implementi hard-negative mining moderno;
10. aggiungi anomalie sintetiche pixel-space;
11. aggiungi branch discriminativa input+reconstruction;
12. aggiungi feature discrepancy;
13. aggiungi anomalie sintetiche feature-space;
14. aggiungi patch nearest-neighbor score;
15. aggiungi teacher-student discrepancy leggera;
16. ottimizzi training con AdamW, warmup, cosine, EMA, clipping;
17. se c'è parte adversarial, stabilizzi con spectral norm, hinge, R1, TTUR;
18. esegui benchmark seri su MVTec AD;
19. fai robustness e few-shot;
20. porti il tutto su MVTec AD 2.

fsfs

21. Le priorità assolute dentro il path B

Se dovessi comprimere tutto al nucleo essenziale, le priorità sono queste:

1. PyTorch + protocollo serio
2. backbone pretrained frozen
3. branch ricostruttiva residuale
4. L1 + MS-SSIM + perceptual
5. latent compactness / one-class bottleneck
6. hard-negative mining modernizzato
7. synthetic anomalies pixel + feature
8. branch discriminativa input+reconstruction
9. feature discrepancy
10. score composito
11. patch nearest-neighbor signal

12. teacher-student discrepancy
13. calibration e thresholding robusto
14. robustness e few-shot
15. MVTec AD / AD2 come benchmark veri

Questa è la versione più completa e cronologica del path B senza lasciare fuori alcun perfezionamento percorribile che sia emerso finora.

Nel prossimo messaggio posso convertirla in una **tabella esecutiva rigidissima** con colonne: **passo, obiettivo, cosa implementare, dipendenze, criterio di successo, ablation minima, priorità.**

Cosa di 2.4 resta da completare più avanti

Non perché manchi infrastruttura, ma perché **i moduli corrispondenti non esistono ancora:**

- reconstruction loss separata
- perceptual / structural loss separata
- latent compactness loss
- discriminative loss
- adversarial loss
- AUROC / AUPRC / F1
- score histograms normali vs anomali
- reconstructions vere
- anomaly maps
- hard negatives visualizzati

Queste voci le completeremo quando costruiremo:

- backbone reale
- branch ricostruttiva reale
- latent core
- branch discriminativa
- scoring
- mining

Sì: nel **path D** i modelli-base da cui partire sono questi.

1. PaDiM

Modella la distribuzione delle **feature patch-level** estratte da un backbone pretrained usando **Gaussiane multivariate** e score tipo Mahalanobis. È il capostipite chiaro della famiglia **feature-modeling patch-based**.

2. PatchCore

Usa feature patch-level da backbone pretrained e costruisce una **memory bank rappresentativa di patch normali**; in test usa una logica **nearest-neighbor / memory retrieval** per score e localizzazione. È il riferimento principale della famiglia **memory-bank patch-based**.

3. DRAEM

È un metodo **reconstruction + discrimination**: una rete ricostruisce una versione “normalizzata” dell’immagine e una seconda rete, vedendo input e ricostruzione, produce

score e mappa d'anomalia. È il riferimento base della famiglia **synthetic-anomaly + discriminative reconstruction**.

4. **RD4AD**

È il riferimento base della famiglia **teacher-student / reverse distillation**: usa un **teacher encoder** e uno **student decoder**, con un **one-class embedding** da cui lo student ricostruisce le feature multiscala del teacher.

5. **EfficientAD**

È una linea **teacher-student efficiente** con una componente globale aggiuntiva di autoencoding per catturare anomalie logiche/globali. Va considerato come baseline base quando oltre all'accuratezza interessa anche **latenza / throughput**.

6. **SimpleNet**

Anche se non lo avevamo messo esplicitamente nell'ultima lista D "corta", concettualmente va incluso tra i modelli-base moderni: appartiene alla famiglia **feature anomaly generation + discriminazione semplice in feature space** ed è molto utile come baseline di riferimento per i metodi che generano pseudo-anomalie nelle feature.

Se vuoi essere rigoroso fino in fondo, il path D si può quindi leggere così, per famiglie:

- **Feature distribution modeling** → **PaDiM**
- **Memory-bank nearest-neighbor** → **PatchCore**
- **Reconstruction + discriminative localization** → **DRAEM**
- **Teacher-student / reverse distillation** → **RD4AD**
- **Teacher-student efficiente / real-time** → **EfficientAD**
- **Feature anomaly generation** → **SimpleNet**

Se vuoi, nel messaggio dopo te li posso ordinare in **quali 3 vale davvero la pena implementare per primi come baseline del path D**.

Top 30 globali

rank	name	cat	seed	val	auroc	auprc	f1	f1@val_thr	fpr95
1	screw_t0_m1a_lfla_oc1_s43	screw	43	1.0000	1.0000	1.0000	1.0000	0.9917	0.0000
2	screw_t0_m1a_lflb_oc0_s43	screw	43	1.0000	1.0000	1.0000	1.0000	1.0000	0.0000
3	screw_t0_m1a_lflb_oc1_s43	screw	43	1.0000	1.0000	1.0000	1.0000	1.0000	0.0000
4	screw_t0_m1b_lfla_oc0_s43	screw	43	1.0000	1.0000	1.0000	1.0000	1.0000	0.0000
5	screw_t0_m1b_lfla_oc1_s43	screw	43	1.0000	1.0000	1.0000	1.0000	1.0000	0.0000
6	screw_t0_m1b_lflb_oc0_s43	screw	43	1.0000	1.0000	1.0000	1.0000	1.0000	0.0000
7	screw_t0_m1b_lflb_oc1_s43	screw	43	1.0000	1.0000	1.0000	1.0000	1.0000	0.0000
8	screw_t1a_m0_lfla_oc0_s43	screw	43	1.0000	1.0000	1.0000	1.0000	1.0000	0.0000
9	screw_t1a_m0_lfla_oc1_s44	screw	44	1.0000	1.0000	1.0000	1.0000	0.9744	0.0000
10	screw_t1a_m0_lflb_oc0_s43	screw	43	1.0000	1.0000	1.0000	1.0000	0.9744	0.0000
11	screw_t1a_m0_lflb_oc0_s44	screw	44	1.0000	1.0000	1.0000	1.0000	0.9655	0.0000
12	screw_t1a_m0_lflb_oc1_s43	screw	43	1.0000	1.0000	1.0000	1.0000	1.0000	0.0000

rank	name	cat	seed	val	auroc	auprc	f1	f1@val_thr	fpr95
13	screw_t1a_m0_lflb_oc1_s44	screw	44	1.0000	1.0000	1.0000	1.0000	0.9916	0.0000
14	screw_t1a_m1a_lfla_oc0_s44	screw	44	1.0000	1.0000	1.0000	1.0000	0.9655	0.0000
15	screw_t1a_m1a_lfla_oc1_s44	screw	44	1.0000	1.0000	1.0000	1.0000	0.9565	0.0000
16	screw_t1a_m1b_lfla_oc0_s43	screw	43	1.0000	1.0000	1.0000	1.0000	0.9655	0.0000
17	screw_t1b_m0_lfla_oc0_s44	screw	44	1.0000	1.0000	1.0000	1.0000	0.9744	0.0000
18	screw_t1b_m0_lflb_oc0_s44	screw	44	1.0000	1.0000	1.0000	1.0000	0.9744	0.0000
19	screw_t1b_m0_lflb_oc1_s44	screw	44	1.0000	1.0000	1.0000	1.0000	0.9744	0.0000
20	screw_t1b_m1a_lfla_oc0_s43	screw	43	1.0000	1.0000	1.0000	1.0000	0.9916	0.0000
21	screw_t1b_m1a_lfla_oc0_s44	screw	44	1.0000	1.0000	1.0000	1.0000	0.9831	0.0000
22	screw_t1b_m1b_lfla_oc0_s44	screw	44	1.0000	1.0000	1.0000	1.0000	0.9831	0.0000
23	tile_t1a_m0_lfla_oc0_s44	tile	44	1.0000	1.0000	1.0000	1.0000	0.9756	0.0000
24	tile_t1a_m1a_lfla_oc0_s43	tile	43	1.0000	1.0000	1.0000	1.0000	0.9880	0.0000
25	tile_t1a_m1a_lfla_oc1_s44	tile	44	1.0000	1.0000	1.0000	1.0000	1.0000	0.0000
26	tile_t1a_m1a_lflb_oc0_s43	tile	43	1.0000	1.0000	1.0000	1.0000	1.0000	0.0000
27	tile_t1a_m1a_lflb_oc1_s43	tile	43	1.0000	1.0000	1.0000	1.0000	0.9880	0.0000
28	tile_t1a_m1a_lflb_oc1_s44	tile	44	1.0000	1.0000	1.0000	1.0000	0.9880	0.0000
29	tile_t1a_m1b_lfla_oc0_s43	tile	43	1.0000	1.0000	1.0000	1.0000	0.9756	0.0000
30	tile_t1a_m1b_lfla_oc1_s44	tile	44	1.0000	1.0000	1.0000	1.0000	1.0000	0.0000

Best per categoria

categoria	best run	val	auroc	auprc	f1	f1@val_thr	fpr95
bottle	bottle_t1a_m1b_lflb_oc1_s43	0.9803	0.9219	0.9740	0.9538	0.9394	0.2000
cable	cable_t0_m1a_lflb_oc1_s43	0.8005	0.6777	0.7856	0.7885	0.7800	0.8621
capsule	capsule_t1b_m1b_lfla_oc1_s43	0.9289	0.7758	0.9150	0.9159	0.9057	0.7500
carpet	carpet_t1a_m0_lfla_oc0_s44	0.9917	0.9524	0.9864	0.9451	0.9348	0.2143
grid	grid_t0_m1b_lflb_oc0_s43	0.9845	0.7053	0.8627	0.8889	0.8358	0.5455
hazelnut	hazelnut_t1a_m1b_lflb_oc0_s44	1.0000	0.9971	0.9985	0.9855	0.9552	0.0000
leather	leather_t1b_m0_lfla_oc0_s43	0.9907	0.8913	0.9624	0.9111	0.8936	0.7500
metal_nut	metal_nut_t1b_m1b_lfla_oc0_s44	0.9203	0.7544	0.9323	0.9091	0.8846	0.6364
pill	pill_t0_m1b_lflb_oc1_s44	0.9569	0.8050	0.9610	0.9342	0.9116	0.7692
screw	screw_t0_m1a_lfla_oc1_s43	1.0000	1.0000	1.0000	1.0000	0.9917	0.0000
tile	tile_t1a_m0_lfla_oc0_s44	1.0000	1.0000	1.0000	1.0000	0.9756	0.0000
toothbrush	toothbrush_t0_m1a_lflb_oc0_s43	1.0000	0.8000	0.9399	0.8889	0.8000	1.0000
transistor	transistor_t0_m1b_lflb_oc1_s43	0.9623	0.8183	0.7465	0.7826	0.6500	0.4000
wood	wood_t1a_m0_lfla_oc0_s43	1.0000	1.0000	1.0000	1.0000	1.0000	0.0000
zipper	zipper_t1a_m0_lfla_oc0_s43	0.9811	0.9125	0.9766	0.9280	0.9194	0.3750

Medie per combinazione

teacher	memory	lf	oc	n	val	auroc	auprc	f1	f1@val_thr	fpr95	std_auroc
tlb	mlb	lflb	oc1	30	0.9333	0.8515	0.9296	0.9142	0.8877	0.4744	0.1271
tla	mla	lfla	oc1	30	0.9243	0.8422	0.9188	0.9130	0.8754	0.4944	0.1410
tlb	m0	lflb	oc0	30	0.9232	0.8274	0.9165	0.9093	0.8792	0.4842	0.1482
tlb	mla	lflb	oc0	30	0.9221	0.8376	0.9225	0.9099	0.8765	0.5245	0.1349
tla	mla	lflb	oc1	30	0.9220	0.8336	0.9182	0.9092	0.8693	0.4698	0.1391
tla	mlb	lflb	oc1	30	0.9220	0.8429	0.9251	0.9114	0.8702	0.4709	0.1416
tlb	mla	lflb	oc1	30	0.9217	0.8466	0.9274	0.9125	0.8804	0.4717	0.1428
tlb	mlb	lflb	oc0	30	0.9213	0.8470	0.9278	0.9096	0.8771	0.5219	0.1309
tla	mla	lflb	oc0	30	0.9208	0.8534	0.9308	0.9116	0.8782	0.4837	0.1233
tla	mlb	lflb	oc0	30	0.9207	0.8424	0.9254	0.9111	0.8693	0.4954	0.1317
tlb	mla	lfla	oc1	30	0.9184	0.8326	0.9184	0.9063	0.8766	0.5347	0.1333
tla	mlb	lfla	oc0	30	0.9172	0.8493	0.9293	0.9100	0.8826	0.4814	0.1315
tla	m0	lfla	oc0	30	0.9167	0.8497	0.9228	0.9159	0.8869	0.4416	0.1395
tlb	mlb	lfla	oc1	30	0.9162	0.8285	0.9194	0.9085	0.8728	0.4898	0.1522
tla	m0	lflb	oc0	30	0.9158	0.8204	0.9105	0.9098	0.8742	0.4790	0.1616
tlb	mlb	lfla	oc0	30	0.9158	0.8511	0.9292	0.9090	0.8790	0.4832	0.1213
tlb	m0	lflb	oc1	30	0.9157	0.8397	0.9247	0.9056	0.8745	0.4966	0.1272
tla	m0	lfla	oc1	30	0.9148	0.8204	0.9078	0.9098	0.8778	0.4594	0.1516
tla	mlb	lfla	oc1	30	0.9142	0.8340	0.9168	0.9097	0.8781	0.4814	0.1385
tla	mla	lfla	oc0	30	0.9142	0.8425	0.9250	0.9114	0.8844	0.4798	0.1355
tla	m0	lflb	oc1	30	0.9142	0.8165	0.9110	0.9042	0.8764	0.5182	0.1534
tlb	m0	lfla	oc0	30	0.9137	0.8269	0.9112	0.9141	0.8710	0.4179	0.1511
tlb	mla	lfla	oc0	30	0.9106	0.8358	0.9187	0.9122	0.8838	0.5034	0.1473
tlb	m0	lfla	oc1	30	0.9051	0.8202	0.9091	0.9072	0.8778	0.4897	0.1574
t0	mlb	lflb	oc0	30	0.9007	0.8014	0.9111	0.8936	0.8633	0.6011	0.1363
t0	mlb	lflb	oc1	30	0.8956	0.7871	0.9048	0.8900	0.8555	0.6688	0.1393
t0	mla	lflb	oc0	30	0.8952	0.7811	0.9027	0.8824	0.8444	0.6606	0.1410
t0	mla	lfla	oc1	30	0.8946	0.7891	0.9054	0.8916	0.8584	0.6369	0.1433
t0	mlb	lfla	oc0	30	0.8912	0.7873	0.9051	0.8879	0.8637	0.6510	0.1378
t0	mla	lfla	oc0	30	0.8910	0.7818	0.9045	0.8859	0.8573	0.6479	0.1434
t0	mla	lflb	oc1	30	0.8893	0.7852	0.9011	0.8875	0.8556	0.6353	0.1363
t0	mlb	lfla	oc1	30	0.8875	0.7708	0.8975	0.8807	0.8537	0.6936	0.1415
t0	m0	lfla	oc0	30	0.8871	0.7894	0.8996	0.8951	0.8659	0.5904	0.1608
t0	m0	lflb	oc1	30	0.8850	0.7720	0.8917	0.8878	0.8511	0.5917	0.1819
t0	m0	lflb	oc0	30	0.8807	0.7821	0.8999	0.8907	0.8626	0.6316	0.1517
t0	m0	lfla	oc1	30	0.8722	0.7755	0.8895	0.8916	0.8612	0.5975	0.1734
tlb	mlb	lf0	oc1	30	0.7322	0.6549	0.8296	0.8774	0.8612	0.7012	0.2588
tlb	mlb	lf0	oc0	30	0.7294	0.6600	0.8311	0.8768	0.8565	0.6941	0.2513
tla	m0	lf0	oc1	30	0.7249	0.6483	0.8223	0.8753	0.8630	0.7060	0.2583
tla	m0	lf0	oc0	30	0.7238	0.6502	0.8266	0.8814	0.8659	0.6892	0.2618
tlb	m0	lf0	oc1	30	0.7230	0.6581	0.8284	0.8801	0.8679	0.6998	0.2600

teacher	memory	lf	oc	n	val	auroc	auprc	f1	f1@val_thr	fpr95	std_auroc
t1a	m1a	lf0	oc0	30	0.7225	0.6433	0.8209	0.8755	0.8631	0.6997	0.2645
t1a	m1a	lf0	oc1	30	0.7222	0.6361	0.8208	0.8761	0.8579	0.7206	0.2804
t1a	m1b	lf0	oc0	30	0.7220	0.6504	0.8256	0.8774	0.8613	0.6896	0.2618
t1b	m1a	lf0	oc1	30	0.7195	0.6362	0.8185	0.8759	0.8625	0.7067	0.2679
t1a	m1b	lf0	oc1	30	0.7184	0.6471	0.8217	0.8769	0.8639	0.7044	0.2669
t1b	m0	lf0	oc0	30	0.7181	0.6360	0.8197	0.8726	0.8592	0.7188	0.2525
t1b	m1a	lf0	oc0	30	0.7174	0.6425	0.8213	0.8778	0.8616	0.6916	0.2765
t0	m0	lf0	oc0	30	0.6724	0.5637	0.7855	0.8610	0.8409	0.7957	0.2694
t0	m1a	lf0	oc0	30	0.6675	0.5672	0.7856	0.8605	0.8438	0.7799	0.2629
t0	m1b	lf0	oc0	30	0.6582	0.5406	0.7737	0.8534	0.8376	0.8522	0.2677
t0	m1a	lf0	oc1	30	0.6571	0.5593	0.7818	0.8592	0.8388	0.8046	0.2677
t0	m0	lf0	oc1	30	0.6527	0.5560	0.7814	0.8561	0.8384	0.8059	0.2665
t0	m1b	lf0	oc1	30	0.6436	0.5347	0.7705	0.8581	0.8380	0.8309	0.2555

Effetto medio delle singole variabili

Teacher

teacher	n	mean_val	mean_auroc	std_auroc	mean_auprc	mean_f1	mean_fpr95
t0	540	0.8123	0.7069	0.2240	0.8606	0.8785	0.6931
t1a	540	0.8528	0.7735	0.2122	0.8877	0.8994	0.5536
t1b	540	0.8532	0.7740	0.2094	0.8890	0.8988	0.5613

Memory

memory	n	mean_val	mean_auroc	std_auroc	mean_auprc	mean_f1	mean_fpr95
m0	540	0.8366	0.7474	0.2195	0.8754	0.8926	0.5896
m1a	540	0.8406	0.7526	0.2182	0.8801	0.8921	0.6081
m1b	540	0.8411	0.7545	0.2150	0.8818	0.8920	0.6103

Learned fusion

lf	n	mean_val	mean_auroc	std_auroc	mean_auprc	mean_f1	mean_fpr95
lf0	540	0.7014	0.6158	0.2678	0.8092	0.8706	0.7384
lf1a	540	0.9058	0.8182	0.1474	0.9127	0.9033	0.5319
lf1b	540	0.9111	0.8204	0.1448	0.9156	0.9028	0.5377

One-class

oc	n	mean_val	mean_auroc	std_auroc	mean_auprc	mean_f1	mean_fpr95
oc0	810	0.8403	0.7541	0.2164	0.8808	0.8928	0.5996
oc1	810	0.8385	0.7489	0.2188	0.8774	0.8917	0.6057

Ranking robusto: media sulle 15 categorie

rank	teacher	memory	lfl	oc	cats	macro_val	macro_auroc	macro_auprc	macro_f1	macro_fpr95	std_cat_auroc	worst_cat_auroc
1	t1a	m1a	lflb	oc0	15	0.9208	0.8534	0.9308	0.9116	0.4837	0.1163	0.6462
2	t1b	m1b	lflb	oc1	15	0.9333	0.8515	0.9296	0.9142	0.4744	0.1191	0.6357
3	t1b	m1b	lfla	oc0	15	0.9158	0.8511	0.9292	0.9091	0.4832	0.1174	0.5986
4	t1a	m0	lfla	oc0	15	0.9167	0.8497	0.9228	0.9159	0.4416	0.1343	0.5967
5	t1a	m1b	lfla	oc0	15	0.9172	0.8493	0.9293	0.9100	0.4814	0.1271	0.6102
6	t1b	m1b	lflb	oc0	15	0.9213	0.8470	0.9278	0.9096	0.5219	0.1247	0.6068
7	t1b	m1a	lflb	oc1	15	0.9217	0.8466	0.9274	0.9125	0.4717	0.1392	0.5722
8	t1a	m1b	lflb	oc1	15	0.9220	0.8429	0.9251	0.9114	0.4709	0.1362	0.6000
9	t1a	m1a	lfla	oc0	15	0.9142	0.8425	0.9250	0.9114	0.4798	0.1220	0.6444
10	t1a	m1b	lflb	oc0	15	0.9207	0.8424	0.9254	0.9111	0.4954	0.1261	0.6672
11	t1a	m1a	lfla	oc1	15	0.9243	0.8422	0.9188	0.9130	0.4944	0.1277	0.5791
12	t1b	m0	lflb	oc1	15	0.9157	0.8397	0.9247	0.9056	0.4966	0.1210	0.6349
13	t1b	m1a	lflb	oc0	15	0.9221	0.8376	0.9225	0.9099	0.5245	0.1259	0.6222
14	t1b	m1a	lfla	oc0	15	0.9106	0.8358	0.9187	0.9122	0.5034	0.1311	0.6091
15	t1a	m1b	lfla	oc1	15	0.9142	0.8340	0.9168	0.9097	0.4814	0.1330	0.6165
16	t1a	m1a	lflb	oc1	15	0.9220	0.8336	0.9182	0.9092	0.4698	0.1355	0.6267
17	t1b	m1a	lfla	oc1	15	0.9184	0.8326	0.9184	0.9063	0.5347	0.1285	0.6049
18	t1b	m1b	lfla	oc1	15	0.9162	0.8286	0.9194	0.9084	0.4898	0.1493	0.5500
19	t1b	m0	lflb	oc0	15	0.9232	0.8274	0.9165	0.9093	0.4842	0.1409	0.5952
20	t1b	m0	lfla	oc0	15	0.9137	0.8269	0.9112	0.9141	0.4179	0.1442	0.5892
21	t1a	m0	lfla	oc1	15	0.9148	0.8204	0.9078	0.9098	0.4594	0.1407	0.5944

rank	teacher	memory	lf	oc	cats	macro_val	macro_auroc	macro_auprc	macro_f1	macro_fpr95	std_cat_auroc	worst_cat_auroc
22	t1a	m0	lf _b	oc ₀	15	0.9158	0.8204	0.9105	0.9098	0.4790	0.1574	0.4953
23	t1b	m0	lf _a	oc ₁	15	0.9051	0.8202	0.9091	0.9072	0.4897	0.1536	0.5690
24	t1a	m0	lf _b	oc ₁	15	0.9142	0.8165	0.9110	0.9042	0.5182	0.1474	0.5944
25	t0	m1b	lf _b	oc ₀	15	0.9007	0.8014	0.9111	0.8936	0.6011	0.1261	0.5523
26	t0	m0	lf _a	oc ₀	15	0.8871	0.7894	0.8996	0.8951	0.5904	0.1408	0.5696
27	t0	m1a	lf _a	oc ₁	15	0.8946	0.7891	0.9054	0.8916	0.6369	0.1349	0.5368
28	t0	m1b	lf _a	oc ₀	15	0.8912	0.7873	0.9051	0.8879	0.6510	0.1304	0.5396
29	t0	m1b	lf _b	oc ₁	15	0.8956	0.7871	0.9048	0.8900	0.6688	0.1326	0.5513
30	t0	m1a	lf _b	oc ₁	15	0.8893	0.7852	0.9011	0.8875	0.6353	0.1297	0.5493
31	t0	m0	lf _b	oc ₀	15	0.8807	0.7821	0.8999	0.8907	0.6316	0.1442	0.5611
32	t0	m1a	lf _a	oc ₀	15	0.8910	0.7818	0.9045	0.8859	0.6479	0.1323	0.5522
33	t0	m1a	lf _b	oc ₀	15	0.8952	0.7811	0.9027	0.8824	0.6606	0.1323	0.5416
34	t0	m0	lf _a	oc ₁	15	0.8722	0.7755	0.8895	0.8916	0.5975	0.1453	0.5416
35	t0	m0	lf _b	oc ₁	15	0.8850	0.7720	0.8917	0.8878	0.5917	0.1667	0.3903
36	t0	m1b	lf _a	oc ₁	15	0.8875	0.7708	0.8975	0.8807	0.6936	0.1318	0.5338
37	t1b	m1b	lf ₀	oc ₀	15	0.7294	0.6600	0.8311	0.8768	0.6941	0.2481	0.3144
38	t1b	m0	lf ₀	oc ₁	15	0.7230	0.6581	0.8284	0.8801	0.6998	0.2530	0.2512
39	t1b	m1b	lf ₀	oc ₁	15	0.7322	0.6549	0.8296	0.8774	0.7012	0.2527	0.2838
40	t1a	m1b	lf ₀	oc ₀	15	0.7220	0.6504	0.8256	0.8774	0.6896	0.2589	0.2404
41	t1a	m0	lf ₀	oc ₀	15	0.7238	0.6502	0.8266	0.8815	0.6892	0.2542	0.2960
42	t1a	m0	lf ₀	oc ₁	15	0.7249	0.6483	0.8223	0.8753	0.7060	0.2513	0.3044
43	t1a	m1b	lf ₀	oc ₁	15	0.7184	0.6471	0.8217	0.8769	0.7044	0.2537	0.3124

rank	teacher	memory	lf	oc	cats	macro_val	macro_auroc	macro_auprc	macro_f1	macro_fpr95	std_cat_auroc	worst_cat_auroc
44	t1a	m1a	lf0	oc0	15	0.7225	0.6433	0.8209	0.8755	0.6997	0.2564	0.2848
45	t1b	m1a	lf0	oc0	15	0.7174	0.6425	0.8213	0.8778	0.6916	0.2604	0.2973
46	t1b	m1a	lf0	oc1	15	0.7195	0.6362	0.8185	0.8759	0.7067	0.2501	0.2884
47	t1a	m1a	lf0	oc1	15	0.7222	0.6361	0.8208	0.8761	0.7206	0.2638	0.2250
48	t1b	m0	lf0	oc0	15	0.7181	0.6360	0.8197	0.8726	0.7188	0.2406	0.3111
49	t0	m1a	lf0	oc0	15	0.6675	0.5672	0.7856	0.8605	0.7799	0.2584	0.2492
50	t0	m0	lf0	oc0	15	0.6724	0.5637	0.7855	0.8610	0.7957	0.2492	0.2606
51	t0	m1a	lf0	oc1	15	0.6571	0.5593	0.7818	0.8592	0.8046	0.2563	0.2477
52	t0	m0	lf0	oc1	15	0.6527	0.5560	0.7814	0.8561	0.8059	0.2503	0.2243
53	t0	m1b	lf0	oc0	15	0.6582	0.5406	0.7736	0.8534	0.8522	0.2625	0.2194
54	t0	m1b	lf0	oc1	15	0.6436	0.5347	0.7705	0.8581	0.8309	0.2378	0.2455

Top 10 combinazioni più robuste

rank	combo				macro_auroc	macro_auprc	macro_f1	std_cat_auroc	worst_cat_auroc
1	t1a	m1a	lf1b	oc0	0.8534	0.9308	0.9116	0.1163	0.6462
2	t1b	m1b	lf1b	oc1	0.8515	0.9296	0.9142	0.1191	0.6357
3	t1b	m1b	lf1a	oc0	0.8511	0.9292	0.9091	0.1174	0.5986
4	t1a	m0	lf1a	oc0	0.8497	0.9228	0.9159	0.1343	0.5967
5	t1a	m1b	lf1a	oc0	0.8493	0.9293	0.9100	0.1271	0.6102
6	t1b	m1b	lf1b	oc0	0.8470	0.9278	0.9096	0.1247	0.6068
7	t1b	m1a	lf1b	oc1	0.8466	0.9274	0.9125	0.1392	0.5722
8	t1a	m1b	lf1b	oc1	0.8429	0.9251	0.9114	0.1362	0.6000
9	t1a	m1a	lf1a	oc0	0.8425	0.9250	0.9114	0.1220	0.6444
10	t1a	m1b	lf1b	oc0	0.8424	0.9254	0.9111	0.1261	0.6672

Riassunto operativo

Le tre più forti da portare avanti sono:

priorità	combo	motivo
1	t1a m1a lf1b oc0	miglior macro-AUROC robusto
2	t1b m1b lf1b oc1	quasi pari, macro-F1 un filo migliore
3	t1b m1b lf1a oc0	terza opzione molto solida

Se vuoi, nel prossimo messaggio ti preparo anche una **tabella finale compatta in CSV/markdown** con solo le top 10 e le best-per-category, pronta da incollare nel repo o nelle note sperimentali.

Conteggi globali

spazio run	
old	1380
new	1920
totale	3300

Space summary globale

spazio	mean val	mean AUROC	mean AUPRC	mean F1	mean FPR@95TPR
old	0.8258	0.7352	0.8710	0.8891	0.6211
new	0.9239	0.8392	0.9237	0.9103	0.5055

Effetto medio globale: teacher

teacher	n	mean val	mean AUROC	std AUROC	mean AUPRC	mean F1	mean FPR@95TPR
t0	540	0.8123	0.7069	0.2240	0.8606	0.8785	0.6931
t1a	900	0.8818	0.7995	0.1894	0.9013	0.9042	0.5294
t1b	900	0.8825	0.8009	0.1864	0.9034	0.9035	0.5404
t1c	480	0.9243	0.8374	0.1402	0.9230	0.9099	0.5056
t1d	480	0.9235	0.8375	0.1400	0.9238	0.9095	0.5164

Effetto medio globale: memory

memory	n	mean val	mean AUROC	std AUROC	mean AUPRC	mean F1	mean FPR@95TPR
m0	540	0.8366	0.7474	0.2195	0.8754	0.8926	0.5896
m1a	900	0.8747	0.7867	0.1959	0.8977	0.8994	0.5684
m1b	900	0.8748	0.7881	0.1926	0.8986	0.8994	0.5721
m1c	480	0.9242	0.8356	0.1418	0.9211	0.9095	0.5078
m1d	480	0.9243	0.8415	0.1349	0.9248	0.9107	0.4981

Effetto medio globale: learned fusion

	lf	n	mean val	mean AUROC	std AUROC	mean AUPRC	mean F1	mean FPR@95TPR
	lf0	540	0.7014	0.6158	0.2678	0.8092	0.8706	0.7384
	lf1a	900	0.9104	0.8250	0.1459	0.9156	0.9057	0.5194
	lf1b	900	0.9157	0.8293	0.1427	0.9190	0.9061	0.5188
	lf1c	480	0.9267	0.8397	0.1372	0.9245	0.9113	0.5036
	lf1d	480	0.9303	0.8365	0.1386	0.9241	0.9093	0.5270

Effetto medio globale: one-class

	oc	n	mean val	mean AUROC	std AUROC	mean AUPRC	mean F1	mean FPR@95TPR
	oc0	2490	0.8973	0.8110	0.1728	0.9095	0.9045	0.5370
	oc1	810	0.8385	0.7489	0.2188	0.8774	0.8917	0.6057

Best per categoria globali

categoria	migliore config	spazio	val	AUROC	AUPRC	F1	F1@thr	FPR@95TPR
bottle	bottle_t1a_m1d_lf1d_oc0_s43	new	0.9949	0.9125	0.9734	0.9524	0.9231	0.7000
cable	cable_t1c_m1b_lf1d_oc0_s43	new	0.8020	0.6829	0.7915	0.7797	0.7719	0.8276
capsule	capsule_t1c_m1c_lf1c_oc0_s43	new	0.9427	0.8121	0.9403	0.9298	0.9091	0.5000
carpet	carpet_t1a_m0_lf1a_oc0_s44	old	0.9917	0.9524	0.9864	0.9451	0.9348	0.2143
grid	grid_t1d_m1a_lf1c_oc0_s43	new	0.9922	0.8589	0.9497	0.8772	0.8615	0.7273
hazelnut	hazelnut_t1b_m1d_lf1b_oc0_s44	new	1.0000	1.0000	1.0000	1.0000	0.9706	0.0000
leather	leather_t1b_m1c_lf1d_oc0_s43	new	0.9924	0.8940	0.9673	0.9091	0.8485	0.8125
metal_nut	metal_nut_t1d_m1a_lf1b_oc0_s44	new	0.9299	0.7021	0.9167	0.8952	0.8627	1.0000
pill	pill_t1b_m1a_lf1d_oc0_s44	new	0.9748	0.8028	0.9512	0.9221	0.8873	0.6923
screw	screw_t0_m1a_lf1a_oc1_s43	old	1.0000	1.0000	1.0000	1.0000	0.9917	0.0000
tile	tile_t1a_m0_lf1a_oc0_s44	old	1.0000	1.0000	1.0000	1.0000	0.9756	0.0000

categoria	migliore config	spazio	val	AUROC	AUPRC	F1	F1@thr	FPR@95TPR
toothbrush	toothbrush_t1c_m1d_lf1d_oc0_s44	new	1.0000	0.8222	0.9162	0.9375	0.7857	0.3333
transistor	transistor_t0_m1b_lf1b_oc1_s43	old	0.9623	0.8183	0.7465	0.7826	0.6500	0.4000
wood	wood_t1a_m0_lf1a_oc0_s43	old	1.0000	1.0000	1.0000	1.0000	1.0000	0.0000
zipper	zipper_t1a_m0_lf1a_oc0_s43	old	0.9811	0.9125	0.9766	0.9280	0.9194	0.3750

Top 20 combinazioni globali per robustezza macro

rank	spazio	teacher	memory	lf	oc	macro AUROC	macro AUPRC	macro F1	std cat AUROC	worst cat AUROC
1	new	t1a	m1a	lf1b	oc0	0.8534	0.9308	0.9116	0.1163	0.6462
2	new	t1b	m1d	lf1d	oc0	0.8533	0.9337	0.9129	0.1201	0.6215
3	new	t1b	m1c	lf1b	oc0	0.8526	0.9293	0.9093	0.1250	0.6331
4	new	t1c	m1c	lf1c	oc0	0.8521	0.9319	0.9136	0.1162	0.6271
5	old	t1b	m1b	lf1b	oc1	0.8515	0.9296	0.9142	0.1191	0.6357
6	new	t1b	m1b	lf1a	oc0	0.8511	0.9292	0.9091	0.1174	0.5986
7	new	t1d	m1d	lf1b	oc0	0.8506	0.9290	0.9120	0.1298	0.6061
8	new	t1d	m1d	lf1c	oc0	0.8506	0.9315	0.9135	0.1210	0.6349
9	old	t1a	m0	lf1a	oc0	0.8497	0.9228	0.9159	0.1343	0.5967
10	new	t1a	m1d	lf1d	oc0	0.8497	0.9282	0.9128	0.1213	0.6441
11	new	t1a	m1b	lf1a	oc0	0.8493	0.9293	0.9100	0.1271	0.6102
12	new	t1a	m1b	lf1d	oc0	0.8471	0.9296	0.9098	0.1164	0.6781
13	new	t1b	m1b	lf1b	oc0	0.8470	0.9278	0.9096	0.1247	0.6068
14	old	t1b	m1a	lf1b	oc1	0.8466	0.9274	0.9125	0.1392	0.5722
15	new	t1c	m1d	lf1b	oc0	0.8461	0.9277	0.9090	0.1252	0.6215
16	new	t1a	m1a	lf1c	oc0	0.8455	0.9218	0.9144	0.1249	0.6462
17	new	t1b	m1b	lf1d	oc0	0.8454	0.9283	0.9116	0.1230	0.6489
18	new	t1a	m1d	lf1c	oc0	0.8453	0.9273	0.9112	0.1248	0.6585
19	new	t1c	m1a	lf1d	oc0	0.8453	0.9338	0.9066	0.1219	0.6278
20	new	t1b	m1d	lf1a	oc0	0.8452	0.9283	0.9079	0.1150	0.5907

Top 20 robuste del solo nuovo spazio

rank	teacher	memory	lf	oc	macro AUROC	macro AUPRC	macro F1	std cat AUROC	worst cat AUROC
1	t1a	m1a	lf1b	oc0	0.8534	0.9308	0.9116	0.1163	0.6462
2	t1b	m1d	lf1d	oc0	0.8533	0.9337	0.9129	0.1201	0.6215

rank	teacher	memory	lf	oc	macro AUROC	macro AUPRC	macro F1	std cat AUROC	worst cat AUROC
3	t1b	m1c	lf1b	oc0	0.8526	0.9293	0.9093	0.1250	0.6331
4	t1c	m1c	lf1c	oc0	0.8521	0.9319	0.9136	0.1162	0.6271
5	t1b	m1b	lf1a	oc0	0.8511	0.9292	0.9091	0.1174	0.5986
6	t1d	m1d	lf1b	oc0	0.8506	0.9290	0.9120	0.1298	0.6061
7	t1d	m1d	lf1c	oc0	0.8506	0.9315	0.9135	0.1210	0.6349
8	t1a	m1d	lf1d	oc0	0.8497	0.9282	0.9128	0.1213	0.6441
9	t1a	m1b	lf1a	oc0	0.8493	0.9293	0.9100	0.1271	0.6102
10	t1a	m1b	lf1d	oc0	0.8471	0.9296	0.9098	0.1164	0.6781
11	t1b	m1b	lf1b	oc0	0.8470	0.9278	0.9096	0.1247	0.6068
12	t1c	m1d	lf1b	oc0	0.8461	0.9277	0.9090	0.1252	0.6215
13	t1a	m1a	lf1c	oc0	0.8455	0.9218	0.9144	0.1249	0.6462
14	t1b	m1b	lf1d	oc0	0.8454	0.9283	0.9116	0.1230	0.6489
15	t1a	m1d	lf1c	oc0	0.8453	0.9273	0.9112	0.1248	0.6585
16	t1c	m1a	lf1d	oc0	0.8453	0.9338	0.9066	0.1219	0.6278
17	t1b	m1d	lf1a	oc0	0.8452	0.9283	0.9079	0.1150	0.5907
18	t1b	m1a	lf1c	oc0	0.8445	0.9255	0.9129	0.1218	0.6376
19	t1d	m1b	lf1d	oc0	0.8444	0.9259	0.9114	0.1265	0.6500
20	t1c	m1a	lf1b	oc0	0.8443	0.9254	0.9099	0.1303	0.6297

Conclusione globale

aspetto	vincitore
spazio medio migliore	new
AUROC medio migliore globale	new
AUPRC medio migliore globale	new
F1 medio migliore globale	new
FPR95 medio migliore globale	new
migliore config robusta assoluta	new: t1a m1a lf1b oc0

Se vuoi, nel prossimo messaggio ti faccio anche la tabella finale più utile di tutte: una **shortlist finale 5 config** con motivazione pratica per scegliere quale mandare ai run con più seed / 16 epoche.

Verdetto

La **config finale migliore** è:

t1d_m1d_lf1c_oc0

perché è prima nel ranking robusto macro e anche prima nella stabilità sui seed.

Numeri chiave della vincitrice

tag	macro_aur oc	macro_aup rc	macro_f 1	macro_fpr9 5	std_cat_aur oc	worst_cat_aur oc
t1d_m1d_lflc_o c0	0.8510	0.9261	0.9149	0.4475	0.1249	0.6301

E sui seed:

tag	mean_seed_auroc	std_seed_auroc	mean_seed_val	std_seed_val
t1d_m1d_lflc_oc0	0.8510	0.0240	0.9261	0.0039

Runner-up

La seconda migliore è:

t1a_m1a_lflb_oc0

Per AUROC macro è vicinissima, ma è leggermente peggiore come worst-category e come stabilità.

tag	macro_aur oc	macro_aup rc	macro_f 1	macro_fpr9 5	std_cat_aur oc	worst_cat_aur oc
t1a_m1a_lflb_o c0	0.8456	0.9235	0.9122	0.4584	0.1367	0.5889

Tutte le tabelle principali

1) Medie per combinazione

tag	n	mean_v al	mean_aur oc	mean_aup rc	mean_f 1	mean_f1th r	mean_fpr9 5	std_auro c
t1d_m1d_lflc_o c0	6 0	0.9261	0.8510	0.9261	0.9149	0.8837	0.4475	0.1405
t1a_m1a_lflb_o c0	6 0	0.9190	0.8456	0.9235	0.9122	0.8812	0.4584	0.1455
t1b_m1c_lflb_o c0	6 0	0.9166	0.8454	0.9225	0.9109	0.8807	0.4831	0.1444
t1b_m1d_lfld_o c0	6 0	0.9268	0.8439	0.9226	0.9109	0.8787	0.4744	0.1371
t1b_m1b_lfla_o c0	6 0	0.9160	0.8436	0.9220	0.9105	0.8758	0.4779	0.1428
t1c_m1c_lflc_oc 0	6 0	0.9249	0.8421	0.9229	0.9092	0.8776	0.5165	0.1354

2) Ranking robusto macro

tag	cat s	macro_ val	macro_ roc	macro_ auc	macro_ f1	macro_ fpr95	std_cat_ auc	worst_cat_ auc
t1d_m1d_lflc_oc0	15	0.9261	0.8510	0.9261	0.9149	0.4475	0.1249	0.6301
t1a_m1a_lflb_oc0	15	0.9190	0.8456	0.9235	0.9122	0.4584	0.1367	0.5889
t1b_m1c_lflb_oc0	15	0.9166	0.8454	0.9225	0.9109	0.4831	0.1327	0.6006
t1b_m1d_lfld_oc0	15	0.9268	0.8439	0.9226	0.9109	0.4744	0.1302	0.6262
t1b_m1b_lfla_oc0	15	0.9160	0.8436	0.9220	0.9105	0.4779	0.1294	0.6062
t1c_m1c_lflc_oc0	15	0.9249	0.8421	0.9229	0.9092	0.5165	0.1283	0.6151

3) Stabilità sui seed

tag	n_ eds	mean_ ed_val	std_ ed_val	mean_ ed_auc	std_ ed_auc	mean_ ed_auprc	std_ ed_auprc	mean_ ed_f1	std_ ed_f1
t1d_m1d_lflc_oc0	4	0.9261	0.0039	0.8510	0.0240	0.9261	0.0139	0.9149	0.0051
t1a_m1a_lflb_oc0	4	0.9190	0.0057	0.8456	0.0279	0.9235	0.0143	0.9122	0.0052
t1b_m1c_lflb_oc0	4	0.9166	0.0069	0.8454	0.0195	0.9225	0.0098	0.9109	0.0060
t1b_m1d_lfld_oc0	4	0.9268	0.0036	0.8439	0.0191	0.9226	0.0129	0.9109	0.0059
t1b_m1b_lfla_oc0	4	0.9160	0.0102	0.8436	0.0252	0.9220	0.0132	0.9105	0.0059
t1c_m1c_lflc_oc0	4	0.9249	0.0082	0.8421	0.0221	0.9229	0.0117	0.9092	0.0068

4) Effetto medio dei fattori

Teacher

teacher	n	mean_val	mean_auc	std_auc	mean_auprc	mean_f1	mean_fpr95
t1a	60	0.9190	0.8456	0.1455	0.9235	0.9122	0.4584
t1b	180	0.9198	0.8443	0.1415	0.9224	0.9108	0.4784
t1c	60	0.9249	0.8421	0.1354	0.9229	0.9092	0.5165
t1d	60	0.9261	0.8510	0.1405	0.9261	0.9149	0.4475

Memory

memory	n	mean_val	mean_auc	std_auc	mean_auprc	mean_f1	mean_fpr95
m1a	60	0.9190	0.8456	0.1455	0.9235	0.9122	0.4584

memory	n	mean_val	mean_auroc	std_auroc	mean_auprc	mean_f1	mean_fpr95
m1b	60	0.9160	0.8436	0.1428	0.9220	0.9105	0.4779
m1c	120	0.9207	0.8438	0.1400	0.9227	0.9100	0.4998
m1d	120	0.9264	0.8474	0.1388	0.9244	0.9129	0.4609

Learned fusion

lf	n	mean_val	mean_auroc	std_auroc	mean_auprc	mean_f1	mean_fpr95
lf1a	60	0.9160	0.8436	0.1428	0.9220	0.9105	0.4779
lf1b	120	0.9178	0.8455	0.1449	0.9230	0.9115	0.4707
lf1c	120	0.9255	0.8465	0.1380	0.9245	0.9120	0.4820
lf1d	60	0.9268	0.8439	0.1371	0.9226	0.9109	0.4744

Lettura pratica

Ci sono tre messaggi forti:

- **Teacher:** t1d è il migliore in media.
- **Memory:** m1d è il migliore in media.
- **Learned fusion:** lf1c è il miglior compromesso, mentre lf1d alza il val ma non l'AUROC medio.

Questo spiega perché la combinazione vincente sia proprio:

t1d_m1d_lf1c_oc0

Best per categoria: chi vince più spesso

Conta vittorie nella tabella “best per categoria”:

- t1d_m1d_lf1c_oc0: hazelnut, leather → **2**
- t1a_m1a_lf1b_oc0: carpet, grid, metal_nut, screw, tile, wood → **6**
- t1b_m1d_lf1d_oc0: cable, pill, transistor, zipper → **4**
- t1c_m1c_lf1c_oc0: bottle, capsule → **2**
- t1b_m1c_lf1b_oc0: toothbrush → **1**

Quindi:

- **più “winner per categoria”:** t1a_m1a_lf1b_oc0
- **migliore media robusta complessiva:** t1d_m1d_lf1c_oc0

Questa è una distinzione importante.

Decisione finale consigliata

Se devi scegliere **una sola config finale di progetto**, io sceglierei:

Finale principale

`t1d_m1d_1f1c_oc0`

perché massimizza il criterio più importante: **performance robusta media su tutte le categorie e tutti i seed.**

Finale di backup / baseline forte

`t1a_m1a_1f1b_oc0`

perché:

- è fortissima
- vince più categorie individuali
- è più “conservativa”
- può essere utile come baseline di riferimento finale

Programma da qui alla fine

Adesso farei così:

1. **fissare la config finale principale** = `t1d_m1d_1f1c_oc0`
2. **fissare la runner-up** = `t1a_m1a_1f1b_oc0`
3. fare un ultimo passaggio solo su queste 2:
 - 15 categorie
 - seed 43 44 45 46
 - **24 o 32 epoche**
4. se `t1d_m1d_1f1c_oc0` resta davanti anche lì, chiudi definitivamente
5. poi fai il pacchetto finale:
 - tabella risultati finali
 - best per categoria
 - macro mean/std
 - confronto baseline vs final model
 - scelta checkpoint/protocollo finale

La mia raccomandazione concreta è: **24 epoche sulle top 2**, non 32 subito.

Se vuoi, nel prossimo messaggio ti scrivo direttamente la mini-grid finale a 24 epoche per queste 2 config.

Mini-report finale

Config scelta

Finale ufficiale: `t1d_m1d_1f1c_oc0`

Interpretazione:

- teacher: t1d
- memory: m1d
- learned fusion: lf1c
- one-class: oc0

Motivazione della scelta

La config è stata scelta perché nella shortlist finale a 16 epoche è risultata:

- **1^a nel ranking robusto macro**
- **1^a nella stabilità sui seed**
- con il miglior compromesso tra:
 - AUROC medio
 - AUPRC medio
 - F1 medio
 - FPR@95TPR più basso
 - bassa variabilità tra categorie
 - buona worst-category performance

Metriche aggregate finali

tag	macro_val	macro_auc	macro_auprc	macro_f1	macro_fpr95	std_cat_auc	worst_cat_auc
t1d_m1d_lf1c_oc0	0.9261	0.8510	0.9261	0.9149	0.4475	0.1249	0.6301

Stabilità sui seed

tag	n_seeds	mean_se_val	std_see_val	mean_see_auroc	std_seed_auroc	mean_see_auprc	std_seed_auprc	mean_see_f1	std_see_f1
t1d_m1d_lf1c_oc0	4	0.9261	0.0039	0.8510	0.0240	0.9261	0.0139	0.9149	0.0051

Confronto con le altre 5 shortlist

tag	mean_auroc	mean_auprc	mean_f1	mean_fpr95
t1d_m1d_lf1c_oc0	0.8510	0.9261	0.9149	0.4475
t1a_m1a_lf1b_oc0	0.8456	0.9235	0.9122	0.4584
t1b_m1c_lf1b_oc0	0.8454	0.9225	0.9109	0.4831
t1b_m1d_lf1d_oc0	0.8439	0.9226	0.9109	0.4744
t1b_m1b_lf1a_oc0	0.8436	0.9220	0.9105	0.4779
t1c_m1c_lf1c_oc0	0.8421	0.9229	0.9092	0.5165

Osservazioni sui fattori

Dalla shortlist finale emerge:

- **teacher t1d** è il migliore in media

- **memory m1d** è il migliore in media
- **learned fusion lf1c** dà il miglior compromesso complessivo
- **one-class score attivo non necessario** nella config finale

Best per categoria della shortlist

category	best_name
bottle	bottle_t1c_m1c_lf1c_oc0_s44
cable	cable_t1b_m1d_lf1d_oc0_s46
capsule	capsule_t1c_m1c_lf1c_oc0_s43
carpet	carpet_t1a_m1a_lf1b_oc0_s46
grid	grid_t1a_m1a_lf1b_oc0_s46
hazelnut	hazelnut_t1d_m1d_lf1c_oc0_s44
leather	leather_t1d_m1d_lf1c_oc0_s43
metal_nut	metal_nut_t1a_m1a_lf1b_oc0_s43
pill	pill_t1b_m1d_lf1d_oc0_s44
screw	screw_t1a_m1a_lf1b_oc0_s46
tile	tile_t1a_m1a_lf1b_oc0_s46
toothbrush	toothbrush_t1b_m1c_lf1b_oc0_s45
transistor	transistor_t1b_m1d_lf1d_oc0_s45
wood	wood_t1a_m1a_lf1b_oc0_s43
zipper	zipper_t1b_m1d_lf1d_oc0_s46

Decisione finale

Per il progetto viene fissata come **config ufficiale**:

t1d_m1d_lf1c_oc0

Nota pratica

Se vuoi ordine nel repo, farei così:

- configs/experiments/final_mvtec_t1d_m1d_lf1c_oc0.yaml
- grid_runs/shortlist_final16/results.csv
- grid_runs/shortlist_final16/summary_report.md

E nel README o nelle note del progetto aggiungerei una riga tipo:

Final selected config: t1d_m1d_lf1c_oc0 chosen from shortlist at 16 epochs over 15 categories $\times$ 4 seeds.

Nel messaggio dopo ti posso anche scrivere un **README finale corto** da incollare nel repo con “how to run / how chosen / final metrics”.

Sì. Ho passato **tutto il file**, non solo i primi blocchi.

Ho estratto programmaticamente tutte le **45 run** del log:

- 15 categorie
- 3 configurazioni memory bank (p1024, p2048, p4096)
- per ciascuna ho preso il blocco finale [Test] e confrontato i risultati.

Il verdetto finale è questo:

migliore complessiva: max_patches=1024
con config:

```
memory_bank:  
  feature_level: layer3  
  max_train_batches: 4  
  max_patches: 1024  
  aggregation: max
```

Perché:

- media Test AUROC più alta tra le tre: **0.8753**
- 2048: **0.8611**
- 4096: **0.8551**

C'è però una cosa interessante:

- come **numero di categorie vinte**, è perfettamente in parità:
 - 1024 vince 5 categorie
 - 2048 vince 5 categorie
 - 4096 vince 5 categorie

Quindi:

- **se vuoi una sola baseline globale**, sceglierei **1024**
- **se vuoi massimizzare categoria per categoria**, non c'è un unico vincitore assoluto

Best per categoria dal file completo:

- 1024 **vince**: bottle, cable, screw, toothbrush, transistor
- 2048 **vince**: hazelnut, metal_nut, tile, wood, zipper
- 4096 **vince**: capsule, carpet, grid, leather, pill

Il dato più importante per la decisione pratica è:

- 1024 è la migliore **in media**
- 1024 è anche quella che vince su **cable** e **transistor**, che erano le categorie più problematiche per voi.

Quindi io farei così:

- baseline finale generale: **layer3 / b4 / p1024 / max**
- se più avanti volete spremere il benchmark, si può fare un override per categoria

Se vuoi, adesso ti preparo direttamente:

1. la **config finale aggiornata**
2. lo **script per rilanciare tutte le categorie** con `p1024`
3. una **tabella completa categoria** → **patch migliore**.